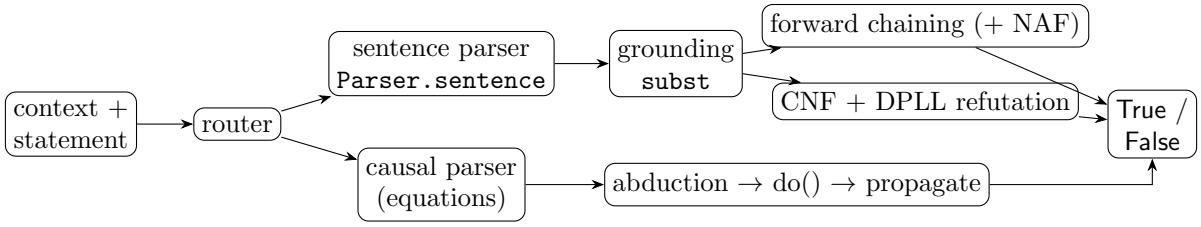
Entailment Checking with a Parser and Classical Inference

Aniruddha Karmokar (P26CS0001) — AI Assignment 2026

1 What the agent does

The agent reads a context (facts and rules in English) and a statement, and answers True if the statement follows from the context, False otherwise. A hand-written parser turns each sentence into a logic formula and a classical algorithm makes the decision. Nothing is learned and no language model is called. Everything is in `reasoning_agent.ipynb`; the entry point is `predict()`, and the notebook runs on any `.jsonl`, `.json` or `.csv` file with `id`, `context` and `statement` set in `TEST_FILE`. The public data folder is optional.



The router has three outcomes. A "... causes ..." context with a "would ... occur" question goes to the causal engine. A context that is a plain rule base (facts, and rules whose conclusion is one positive statement) goes to forward chaining. Everything else (*or*, *either*, *no*, negative facts or conclusions, quantified queries) goes to DPLL.

2 Turning sentences into logic

Predicates and terms. A simple sentence becomes `predicate(subject)`. The predicate is the verb phrase after `clean` and `pred`: lower case, no articles, no *is/are/do/does*, each word cut to a rough stem by `stem`. So "is not dancing gracefully" and "dances gracefully" both give `danc gracefully`, the first one negated. The object stays inside the predicate ("likes the cat" → `lik cat`), but inside a rule an object written as *something* or *it* becomes the variable `X`, so relations such as "the cat chases it" still work.

Sentence	Internal form (printed by the notebook)
If someone is smart and kind, then they are successful.	$smart(X) \wedge kind(X) \rightarrow successful(X)$
All students who play chess are clever.	$student(X) \wedge play\ chess(X) \rightarrow clever(X)$
Some birds can not fly.	$\exists X\,(bird(X) \wedge \neg can\ fly(X))$
Rina is either a student or a teacher.	$student(rina) \oplus teacher(rina)$
Vacuums suck.	$vacuum(X) \rightarrow suck(X)$
Red, round things are big.	$red(X) \wedge round(X) \rightarrow big(X)$
If something likes the cat and it is not red then the cat chases it.	$lik\ cat(X) \wedge \neg red(X) \rightarrow chas\ X(cat)$

Table 1: Parser output for a few sentences (`show(Parser().sentence(s))`).

Sentence patterns. `Parser.sentence` tries, in order: *if and only if*; conditionals (*if, when, assuming, given that, based on*, with *then*, a comma, or nothing before *they/it*); *only if*; quantifiers (*all, every, no, some, not all, there is*); *someone/everyone (who)*; generic subjects (*a/an N*, *people who*, bare plurals); and a plain fact. `vp` handles *not*, *it is not true that*, *either/or*, *or both*, *neither/nor*, *and* and *who*. A sentence that matches nothing is skipped, not guessed.

Removing variables. A formula with `X` is copied for every individual named in the text (as subject or object) plus one extra individual `new`. "Some" premises get a fresh constant. A universal query is proved for `new`: if it holds for an individual with no special properties, it holds for all. Cause–effect text is read separately: "Pex or Zurn causes not Zlim" gives the equation $Zlim := \neg(Pex \vee Zurn)$, "We observed V" is evidence, and "not X instead of X" is $do(X = 0)$.

3 How a decision is made

Rule bases (`program`, `strata`, `forward_chain`). The ground formulas are split into facts and rules *positive premises, negated premises* $\Rightarrow$ *conclusion.* A negated premise "not p" holds if p cannot be derived (negation as failure). To make this safe, `strata` gives every statement a level that is above the level of anything it needs to be false. Rules are then applied level by level until nothing new appears, so a "not p" test only runs after every rule that could produce p is finished. A negated query is True when its statement is not derived.

When the world is closed. Interpreting "not derived" as "false" is accurate for RuleTaker-like theories but incorrect for FOLIO, where numerous elements are merely unknown. Therefore, negation as failure is only applied when all sentences have been processed and each predicate consists of either a single word or a word along with a named individual, such as red or lik cat. In other cases, the issue is handled by DPLL, which applies standard (open-world) logic.

Everything else (`dpll`). The statement is entailed exactly when $KB \wedge \neg q$ has no model. After `nnf` and `cnf`, DPLL searches depth first: it applies unit clauses, and when none are left it tries a literal from the shortest clause as true, then false. If the KB alone has no model (a parsing mistake), the answer is False, so a contradiction is never used to prove everything.

"Would Y occur" questions (`causal_predict`). For every assignment of the root variables, `solve(u, {})` computes the real world and keeps it if it matches the observations (abduction); `solve(u, do)` then fixes the intervened variables and recomputes the rest. The answer is True only if the target is true in every kept world.

4 Worked examples

Printed by `explain()` in the notebook, redrawn here as small derivations.

Brief example. *Bob is successful* $\Rightarrow$ True: `young(bob)` [given] $\Rightarrow$ `smart(bob)`; `smart(bob)` $\wedge$ `kind(bob)` [given] $\Rightarrow$ `successful(bob)`.

RuleTaker with negation (`std_ruletaker_0391`, gold True). Query *Fiona is kind.*

```
blu(fiona)                                          [given]
blu(fiona) and not round(fiona) => green(fiona)    [round(fiona) is never derived]
green(fiona) => kind(fiona)                         [If Fiona is green then Fiona is kind]
```

FOLIO with DPLL (`folio_train_715`). "No plants are fungi. Mushrooms are fungi." Query *No plants are mushrooms.* On `new` the clauses are $\{\neg plant, \neg fungi\}$, $\{\neg mushroom, fungi\}$ and, from $\neg q$, $\{plant\}$, $\{mushroom\}$. Unit propagation gives $plant \Rightarrow \neg fungi \Rightarrow \neg mushroom$, which clashes with $\{mushroom\}$: no model, so True.

CounterBench (`counterbench_alpha_V1_209`). "Glent or Razz causes Pex, Pex causes Zurn, Zurn causes Zlim. We observed Razz." Query: *Would Zlim occur if not Glent instead of Glent?* Equations $pex := glent \vee razz$, $zurn := pex$, $zlim := zurn$. The kept worlds both have $razz = 1$; with $do(glent = 0)$ both give $zlim = 1$, so True (gold True).

5 Results on the public datasets

Gold labels are only used for scoring. FOLIO *True* is True; *False/Uncertain* are False. RuleTaker is the linked `jise/ruletaker` release, evaluated on all of its rows. Because that release has no negation, I also generated 1000 theories in the *original* RuleTaker style (someone/something rules, "does not", relations, "Red, round things are ...") and labelled them with ProbLog, the engine RuleTaker itself uses, under the closed-world assumption (`data/make_ruletaker_standard.py`). This set was generated after development with a new seed.

Split	Queries	Accuracy (%)	Always-False (%)	Time (s)
RuleTaker test / ood_test	250 / 375	97.60 / 99.47	37.60 / 65.07	0.14 / 0.20
FOLIO validation	203	67.98	64.53	0.14
CounterBench (all public)	1200	94.17	50.42	0.20
RuleTaker original style (generated)	1000	100.00	50.70	0.62
Official splits / all evaluation rows	2028 / 3028	92.95 / 95.28		
RuleTaker / FOLIO train (not held out)	1000 / 1001	98.30 / 62.84	40.60 / 61.24	0.46 / 0.78

Table 2: Accuracy and time. All 5029 queries take about 2.5 s (0.5 ms per query).

`world`	RuleTaker orig. style	RuleTaker test	FOLIO val.	FOLIO train
open (no negation as failure)	66.30	97.60	67.98	62.94
closed (always)	100.00	97.60	64.04	59.24
auto (controlled vocabulary)	**100.00**	**97.60**	**67.98**	62.84

Table 3: Why the closed-world test is needed (accuracy %, notebook cell after the evaluation).

Other checks. On CounterBench, also treating "instead of X" as evidence that X was true (`strict=True`) gives 90.42% instead of 94.17%, so only "We observed" is used as evidence. By type: basic 100, joint 96, nested 96, backdoor 90, conditional 88 (%). RuleTaker test+OOD by depth 1–5: 100, 94.7, 99.0, 100, 98.4 (%).

Where the errors come from. *RuleTaker:* all 8 errors are True answers to queries labelled False, each checkable in one or two steps; e.g. in `ruletaker_test_0143` "The chef is an engineer" and "Given that the chef is an engineer, it follows that the plane travels far" are given, but "The plane travels far" is labelled False. *CounterBench:* the 70 errors come from 14 graph templates (5 renamed copies each); some gold answers cannot be obtained from the equations under any reading, and 5 rows have a cycle. *FOLIO:* no False or Uncertain validation query is answered True, but only 7 of 72 True queries are proved, because premises and queries often word the same idea differently.

6 What does not work yet

Predicates consist of a single subject, which means that a rule connecting two variables (such as "if X likes Y, then Y likes X") cannot be formulated. A parsed sentence can still convey a different meaning, synonyms are not recognized, and False does not clarify whether the claim has been disproven or simply remains uncertain. The limitation lies in the English parser rather than the reasoning; the three engines are precise with the input they receive.